GLOBAL SUPERSTORE MANAGEMENT

Data Management for Analytics

Final Report

Professor: Venkat Krishnamurthy

Group-16

Laawanyaa Sai Thota(002208176)

Udhay Chityala(002830533)

Submission Date: 12/06/2023

Business Problem

The goal of this project is to develop a relational database for a global superstore where customers can purchase a wide range of products, including electronics, home essentials, furniture, personal care items, clothing, and groceries. Users will register through an online application, which will capture their personal information, order history, and payment details.

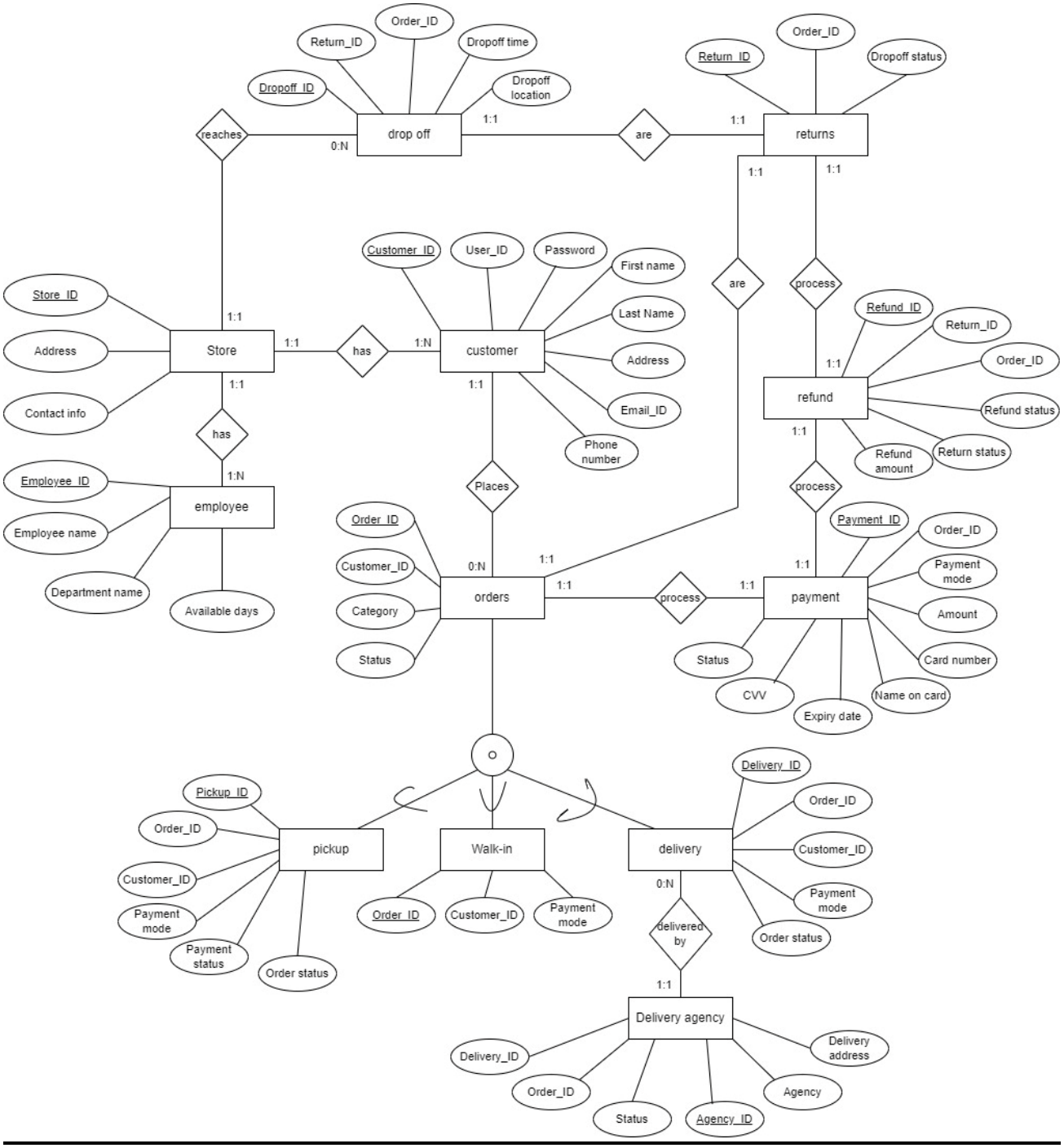
Customers will have three options for order fulfilment: pickup, delivery, or in-store purchase. When choosing pickup, customers can decide to pay either before or upon collecting their orders. When selecting delivery, payment is required at the time of order placement, and a delivery service such as FedEx or UPS will handle the shipment.

The system will also maintain records of all the employees working for the store, including their ID, name, job titles. If customers are unsatisfied with a product, they can request a return through the online application, with the choice of sending the items back via a courier service or returning it to the physical store. The refund will be processed within 5-7 business days and credited to the customer's account. All this data can be used to analyze store performance like top performing store, top customers, high value customers, their reason for being high value (due to AOV or OPC or both?) etc..

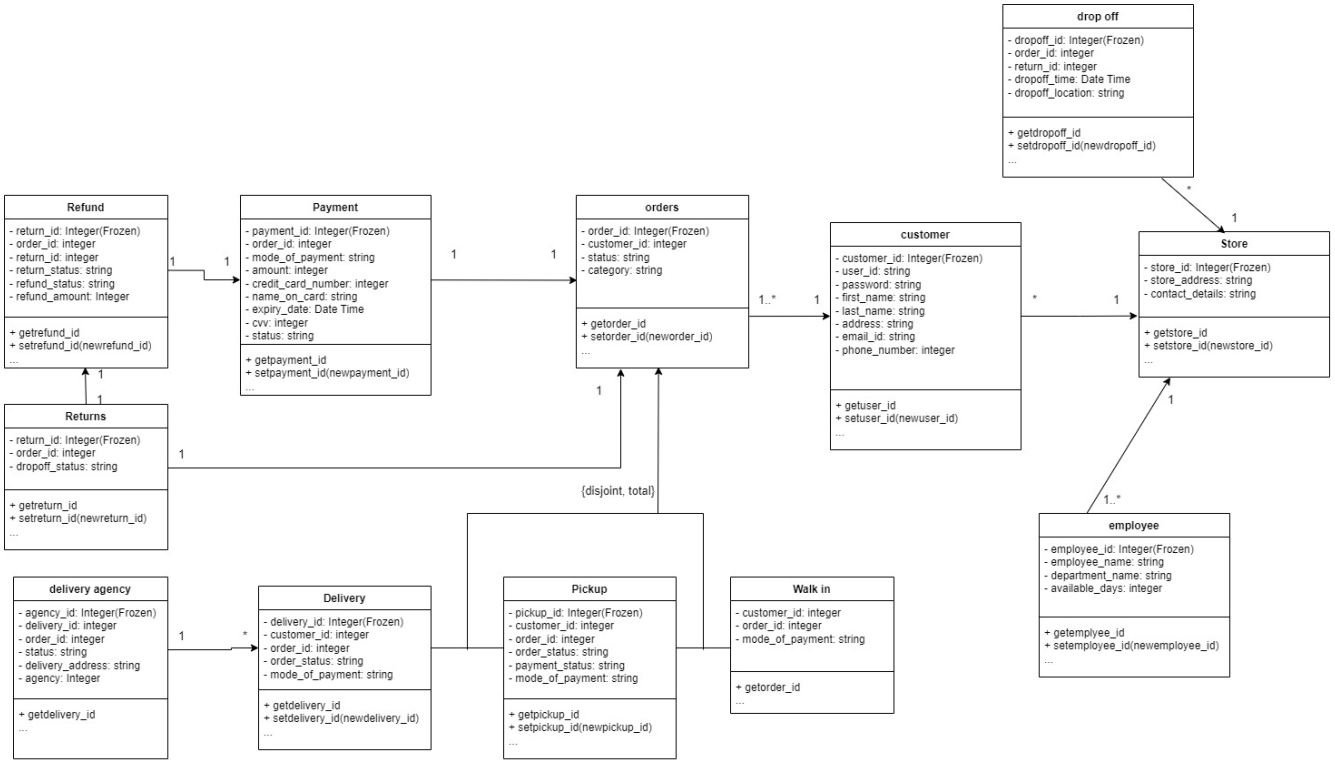
Assumptions:

1. A customer must have one account only
2. A customer cannot order without registering in the online application
3. An order can be assigned to only one delivery agent but a delivery agent can deliver multiple orders
4. In case of returns, order must be returned completely (no partial returns)
5. Only one mode of payment can be used for a specific order

EER Diagram


Order_ID
Return_ID
Dropoff time
Dropoff_ID
Dropoff location
drop off
reaches
0:N
1:1
are
1:1
returns
Return_ID
Order_ID
Dropoff status
1:1
1:1
Customer_ID
User_ID
Password
First name
Last Name
Address
Email_ID
Phone number
customer
1:N
has
1:1
Store_ID
Address
Contact info
Store
1:1
1:1
has
1:N
Employee_ID
Employee name
Department name
Available days
employee
are
process
Refund_ID
Return_ID
Order_ID
Refund status
Return status
Refund amount
refund
1:1
1:1
Places
0:N
1:1
Order_ID
Customer_ID
Category
Status
orders
process
1:1
1:1
Payment_ID
Order_ID
Payment mode
Amount
Card number
Name on card
Expiry date
CVV
Status
payment
Pickup_ID
Order_ID
Customer_ID
Payment mode
Payment status
Order status
pickup
Walk-in
Order_ID
Customer_ID
Payment mode
Delivery_ID
Order_ID
Customer_ID
Payment mode
Order status
delivery
0:N
delivered by
1:1
Delivery_ID
Order_ID
Status
Agency_ID
Agency
Delivery address
Delivery agency

UML Diagram



Mapping Conceptual model to Relational model

Table Name – **BOLD**, Primary Key – Underlined, Foreign Key - *Italics*

Store (Store_ID, Store_Address, Contact_Details)

Customer (Customer_ID, User_ID, Password, First_Name, Last_Name, Address,

Email_ID, Phone_Number)

Orders (Order_ID, Order_Category, Payment_Status, *Customer_ID*)

- Customer_ID (Foreign key) refers to Customer_ID in the relation Customer: Null not allowed

Walk-In (Order_ID, Mode_of_Payment)

Pickup (Order_ID, Pickup_ID, Order_Status, Mode_of_Payment)

Delivery (Delivery_ID, Order_ID, Order_Status, Mode_of_Payment, *Agency_ID*)

- Agency_ID (Foreign key) refers to Location in the relation Delivery_Agency: Null not allowed

Delivery_Agency (Agency_ID, *Order_ID*, Status, Delivery_Address)

- Order_ID (Foreign key) refers to Order_ID in the relation Orders: Null not allowed

Agency (Agency_ID, Agency_Name)

Payment (Payment_ID, Payment_Status, Mode_of_Payment, CVV, Expiry_Date, Name_on_Card, Card_number, amount, mode_of_payment, *Order_ID*)

- Order_ID (Foreign key) refers to Order_ID in the relation Orders: Null not allowed

Refund (Refund_ID, Refund_Status, Refund_Amount, *Order_ID*, *Return_ID*)

- Order_ID (Foreign key) refers to Order_ID in the relation Orders: Null not allowed
- Return_ID (Foreign key) refers to Return_ID in the relation Returns: Null not allowed

Returns (Return_ID, Dropoff_Status, *Order_ID*, *Dropoff_ID*)

- Order_ID (Foreign key) refers to Order_ID in the relation Orders: Null not allowed
- Dropoff_ID (Foreign key) refers to Dropoff_ID in the relation Orders: Null not allowed

Dropoff (Dropoff_ID, Dropoff_time, Dropoff_location, *Order_ID*)

- Order_ID (Foreign key) refers to Order_ID in the relation Orders: Null not allowed

Employee (Employee_ID, Employee_Name, Department_Name, Available_Days, *Store_ID*)

- Store_ID (Foreign key) refers to Store_ID in the relation Store: Null not allowed

SQL Implementation

Query 1) Find all delivery orders from orders table

Analytical Purpose: Manager wanted to look at the delivery orders to view their payment status, store and customerid

```
SELECT *

FROM orders

WHERE Order_Category = 'delivery';
```

Order_ID	Order_Category	Payment_Status	Customer_ID	Store_ID	Orderdate
2	delivery	Pending	2	2	2023-11-09
11	delivery	Pending	11	1	2023-11-17
13	delivery	Completed	13	3	2023-10-29
15	delivery	Pending	15	5	2023-10-24
20	delivery	Completed	20	5	2023-11-15
21	delivery	Completed	21	1	2023-11-14
22	delivery	Pending	22	2	2023-11-05
34	delivery	Completed	34	5	2023-11-02
35	delivery	Completed	35	3	2023-11-06
37	delivery	Pending	37	2	2023-11-10
39	delivery	Pending	39	5	2023-11-01
44	delivery	Pending	44	5	2023-10-25

Query 2) Find the number of orders in each store

Analytical Purpose: Identifying the orders per store to analyze the store performance and plan on any marketing campaigns to promote sales in low performing stores

```
SELECT Store_ID,

        Count(*) AS No_of_orders

FROM orders

GROUP BY Store_ID;
```

Store_ID	No_of_orders
1	20
2	20
3	36
4	4
5	20

Query 3) Find customers who made purchases using Credit Card

Analytical Purpose: Identifying the customers who made purchases with credit card to approve their transaction after adding tip (if any).

```
SELECT DISTINCT C.Customer_ID, C.First_Name, C.Last_Name
FROM Customer C
LEFT OUTER JOIN Orders O ON C.Customer_ID = O.Customer_ID
LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
WHERE Mode_of_Payment = "Credit card"
ORDER BY Customer_ID;
```

Customer_ID	First_Name	Last_Name
5	Maudie	Wyman
13	Winfield	Konopelski
16	Dasia	Bins
17	Alexys	Purdy
21	Lois	Kuvalis
22	Magnolia	O'Kon
25	Judson	Reynolds
27	Winston	Parker
29	Keanu	Howe
30	Lewis	Pouros
33	Sophia	Will
34	Laurence	Borer

Query 4) Find the total amount spent by each customer in descending order of their spending

Analytical Purpose: Identifying and rewarding the top 3-5 high-value customers with personalized gift cards in celebration of Thanksgiving

```
SELECT C.Customer_ID, C.First_Name, C.Last_Name, SUM(P.Amount) AS Total_spend_$
FROM orders O
LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
LEFT OUTER JOIN customer C ON O.Customer_ID = C.Customer_ID
GROUP BY C.Customer_ID,C.First_Name,C.Last_Name
ORDER BY Total_spend_$ DESC;
```

Customer_ID	First_Name	Last_Name	Total_spend_$
39	Bette	Kutch	2815
45	Mercedes	Monahan	2342
21	Lois	Kuvalis	2293
64	Lillie	Hilpert	2120
5	Maudie	Wyman	2048
13	Winfield	Konopelski	2002
25	Judson	Reynolds	1960
35	Deja	Morar	1773
9	Brady	Schiller	1696
6	Lilian	Durgan	1646
17	Alexys	Purdy	1641
29	Keanu	Howe	1440

Query 5) Find the store with highest revenue

Analytical Purpose: Determining the top-performing store based on revenue to award the coveted 'Super Store!!' recognition as part of the company's monthly awards to inspire and motivate the team.

```
SELECT S.Store_ID, SUM(P.Amount) AS Total_revenue_$
FROM orders O
LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
LEFT OUTER JOIN store S ON O.Store_ID = S.Store_ID
GROUP BY S.Store_ID
HAVING Total_revenue_$ = (SELECT MAX(Total_revenue_$)
                          FROM (SELECT S.Store_ID, SUM(P.Amount) AS Total_revenue_$
                                FROM orders O
                                LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
                                LEFT OUTER JOIN store S ON O.Store_ID = S.Store_ID
                                GROUP BY S.Store_ID ) AS Store_rev);
```

Store_ID	Total_revenue_$
3	22143

Query 6) Find all customers with atleast 2 orders

Analytical Purpose: Identifying the repeating customers to

```
SELECT C.Customer_ID, C.First_Name, C.Last_Name
FROM Customer C
WHERE EXISTS (SELECT O.customer_ID,count(*) as no_of_orders
                FROM Orders O
                WHERE C.Customer_ID = O.Customer_ID
                GROUP BY O.customer_ID
                HAVING no_of_orders>=2);
```

Customer_ID	First_Name	Last_Name
5	Maudie	Wyman
6	Lilian	Durgan
9	Brady	Schiller
11	Mekhi	Gleason
13	Winfield	Konopelski
15	Vincenza	Sanford
16	Dasia	Bins
17	Alexys	Purdy
21	Lois	Kuvalis
22	Magnolia	O'Kon
25	Judson	Reynolds
29	Keanu	Howe

Query 7) Find distribution of revenue by payment mode

Analytical Purpose: Identifying the share of each payment method so that we can contact a credit/debit card vendor for special offers for using their credit/debit card which boosts the sales

```
SELECT Mode_Of_Payment,
        SUM(amount) AS Revenue,
        ROUND(100*SUM(amount)/(SELECT SUM(amount) FROM Payment),2)
        AS 'Revenue_Share(%)'
FROM Payment
GROUP BY Mode_Of_Payment
ORDER BY Revenue
```

	Mode_Of_Payment	Revenue	Revenue_Share(%)
▶	Cash	4636	8.13
	Paypal	11926	20.91
	Credit card	12688	22.24
	Applepay	12891	22.6
	Debit card	14906	26.13

Query 8) Find Top 10 High value customers and their average order value (AOV) & number of orders (OPC)

Analytical Purpose: Further analyzing the top-10 high LTR ($) customers to evaluate if the LTR is due to few high value orders or due to several low value orders

```
SELECT Customer_ID,
        First_Name,
        Last_Name,
        ROUND(AOV,2) AS 'AOV($)',
        Total_rev AS 'LTR($)',
        No_of_orders
FROM (SELECT C.Customer_ID, C.First_Name, C.Last_Name, SUM(P.amount) AS Total_rev,
              AVG(P.amount) AS AOV,COUNT(p.amount) AS No_of_orders,
              RANK() OVER (ORDER BY SUM(P.amount) DESC, AVG(P.amount) DESC) AS Cust_Rank
       FROM Customer C
       LEFT OUTER JOIN Orders O ON C.Customer_ID = O.Customer_ID
       LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
       GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
      ) RankedCustomers
WHERE Cust_Rank <= 10;
```

	Customer_ID	First_Name	Last_Name	LTR($)	AOV($)	No_of_orders
▶	39	Bette	Kutch	2815	938.33	3
	45	Mercedes	Monahan	2342	780.67	3
	21	Lois	Kuvalis	2293	764.33	3
	64	Lillie	Hilpert	2120	706.67	3
	5	Maudie	Wyman	2048	682.67	3
	13	Winfield	Konopelski	2002	500.5	4
	25	Judson	Reynolds	1960	653.33	3
	35	Deja	Morar	1773	886.5	2
	9	Brady	Schiller	1696	848	2
	6	Lilian	Durgan	1646	548.67	3

Query 9) Find Top 2 Payment methods in each store by sales ($)

Analytical Purpose: To find the sales($) by payment method based on which our manager can take a call on whether to purchase an additional cash counting machine (if cash has major share in the sales($))

```
SELECT Store_ID, Mode_of_Payment, Cat_rev AS 'Category_rev($)',
        (SELECT SUM(amount)
        FROM Orders O
        LEFT OUTER JOIN Payment P on O.Order_ID = P.Order_ID
        WHERE O.Store_ID = p1.store_ID) AS 'Store_rev($)',
        ROUND(100*Cat_rev/(SELECT SUM(amount)
                           FROM Orders O
                           LEFT OUTER JOIN Payment P on O.Order_ID = P.Order_ID
                           WHERE O.Store_ID = p1.store_ID) ,2) AS 'Rev_Share_in_store(%)'
FROM (SELECT O.Store_ID,
             P.Mode_of_Payment ,
             SUM(P.amount) AS Cat_rev,
             RANK() OVER (PARTITION BY O.Store_ID ORDER BY SUM(P.amount) DESC) AS
             Payment_Rank
        FROM Orders O
        LEFT OUTER JOIN Payment P ON O.Order_ID = P.Order_ID
        GROUP BY O.Store_ID,p.Mode_of_Payment) AS P1
WHERE Payment_Rank<=2
ORDER BY Store_ID,Cat_rev DESC;
```

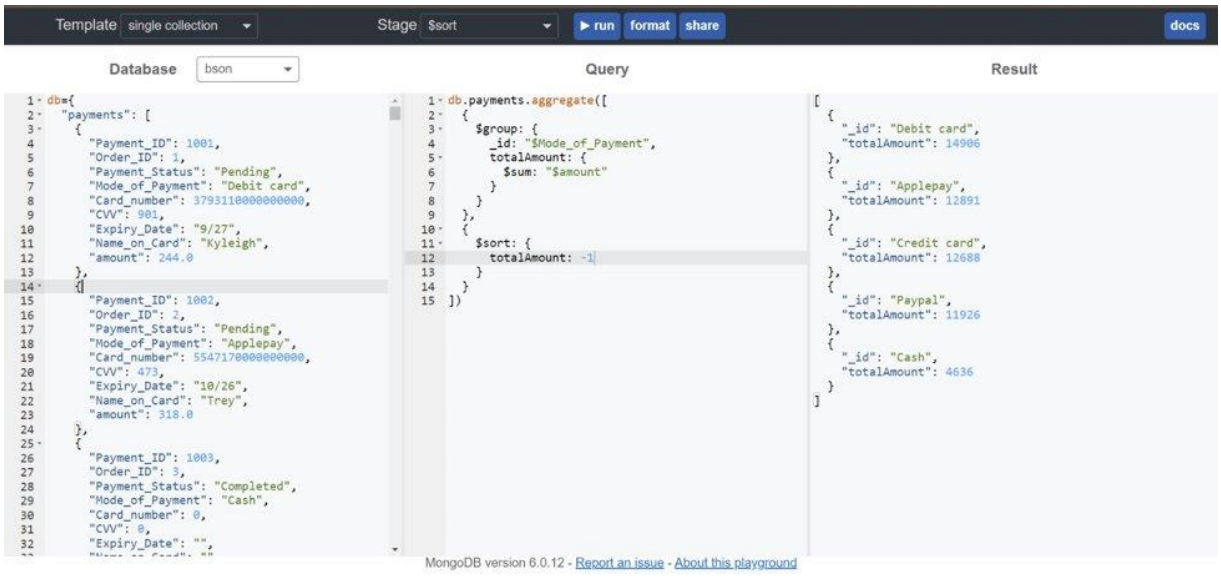
	Store_ID	Mode_of_Payment	Category_rev($)	Store_rev($)	Rev_Share_in_store(%)
▶	1	Debit card	3293	9972	33.02
	1	Credit card	2482	9972	24.89
	2	Applepay	3797	12268	30.95
	2	Credit card	2694	12268	21.96
	3	Applepay	5836	22143	26.36
	3	Debit card	5722	22143	25.84
	4	Debit card	1492	1922	77.63
	4	Cash	430	1922	22.37
	5	Paypal	4259	10742	39.65
	5	Credit card	2571	10742	23.93

NoSQL Implementation

/* Finding the Sales ($) through each payment method */

Analytical Purpose: Identifying the share of each payment method so that we can contact a credit/debit card vendor for special offers for using their credit/debit card which boosts the sales

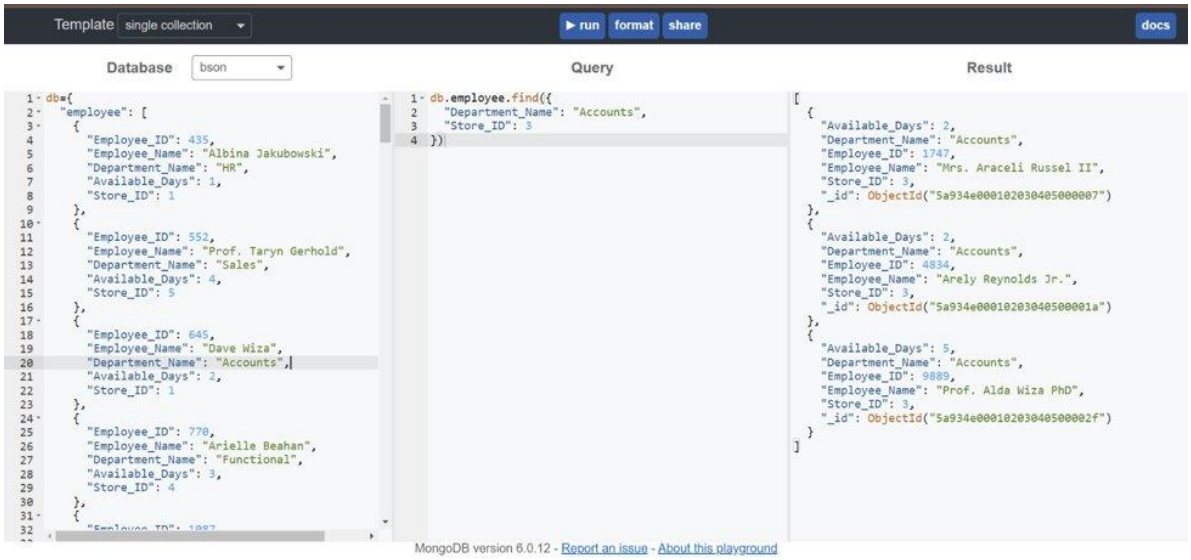
```
db.payments.aggregate([
{
        $group: {
        _id: “$Mode_of_Payment”,
        totalAmount: {$sum:”$amount”}
        }
},
{$sort: {totalAmount: -1}}
])
```



/* Identifying the employees in accounts department in store 3*/

Purpose: Need to do background verification of the employees in store 3 accounts department.

```
db.employee.find({
        “Department_Name”:“Accounts”,
        “Store_ID”: 3
        })
```



/* Identifying the payment details of completed payments with amount > $750*/

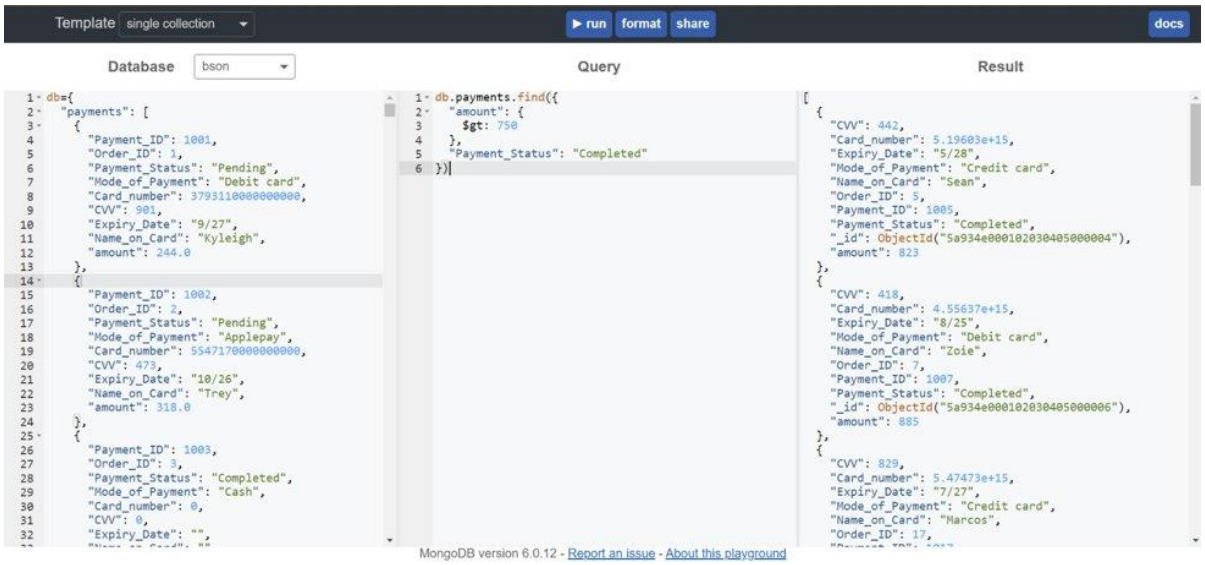
Purpose: Need to manually verify transactions with huge amount

db.payments.find({

“amount”: {$gt: 750},

“Payment_Status”: “Completed”

})



Connecting Python with MySQL

In [3]:
```
pip install mysql-connector-python
```

```
Requirement already satisfied: mysql-connector-python in c:\users\admin\anaconda3\lib\site-packages (8.2.0)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: protobuf<=4.21.12,>=4.21.1 in c:\users\admin\anaconda3\lib\site-packages (from mysql-connector-python) (4.21.12)
```

In [53]:
```
# Importing module
import mysql.connector
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt
```

In [68]:
```
import mysql.connector
from mysql.connector import errorcode
try:
  cnx = mysql.connector.connect(host='127.0.0.1',
                                database='superstore',
                                user='root',
                                password='Sri@199789')
except mysql.connector.Error as err:
  if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
    print("Something is wrong with your user name or password")
  elif err.errno == errorcode.ER_BAD_DB_ERROR:
    print("Database does not exist")
  else:
    print(err)
else:
    print("Sucessfully connected")
    mycursor = cnx.cursor()
```

```
Sucessfully connected
```

Find distribution of revenue by payment mode using python

Analytical Purpose: Identifying the share of each payment method so that we can contact a credit/debit card vendor for special offers for using their credit/debit card which boosts the sales

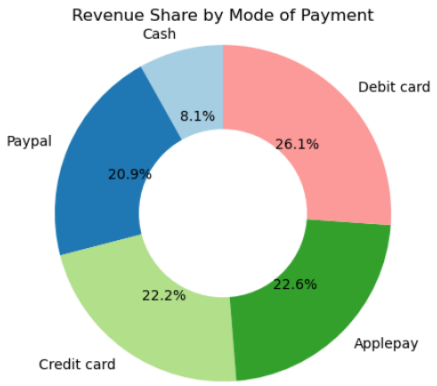
In [69]:
```
mycursor.execute("SELECT Mode_Of_Payment, SUM(amount) AS Revenue, ROUND(100 * SUM(amount) / (SELECT SUM(amount) FROM Payment), 2)
result1 = pd.DataFrame(mycursor.fetchall(), columns=['Mode_Of_Payment', 'Revenue', 'Revenue_Share(%)'])
```

In [70]:
```
print(result1)
```

```
  Mode_Of_Payment  Revenue  Revenue_Share(%)
0            Cash   4636.0             8.13
1          Paypal  11926.0            20.91
2     Credit card  12688.0            22.24
3       Applepay  12891.0            22.60
4      Debit card  14906.0            26.13
```

Visualizing the output using donut chart

In [71]:
```
# Visualize the data with a donut chart
labels = result1['Mode_Of_Payment']
sizes = result1['Revenue_Share(%)']
colors = plt.cm.Paired(range(len(labels)))

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90, colors=colors)
centre_circle = plt.Circle((0, 0), 0.50, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle.
plt.title('Revenue Share by Mode of Payment')
plt.show()
```



Find the total amount spent by each customer in descending order of their spending using python

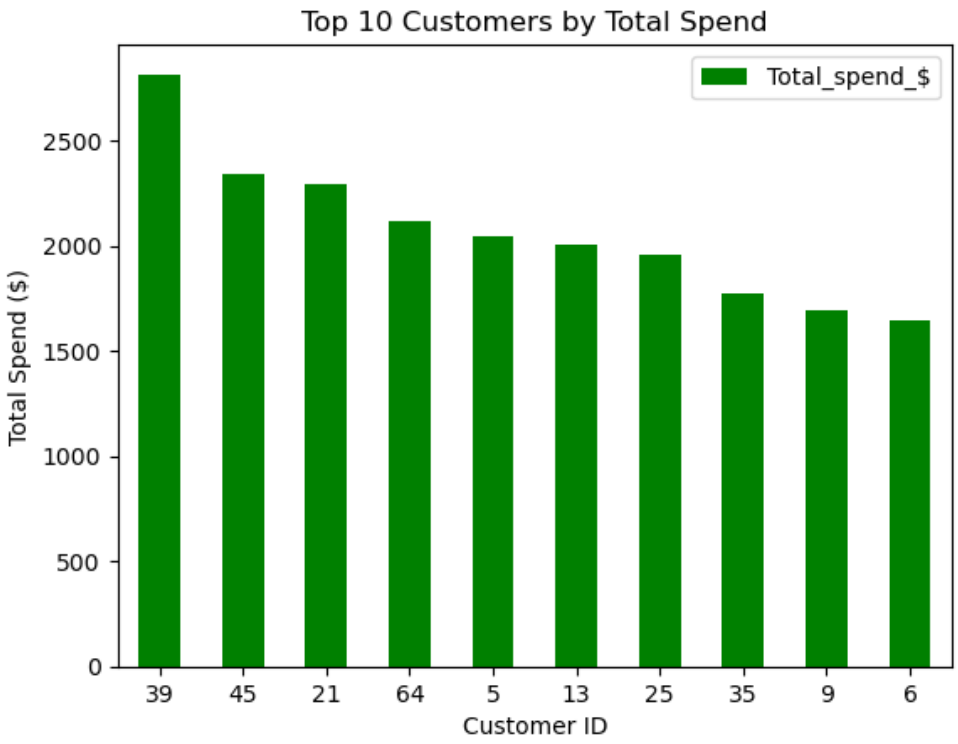
Analytical Purpose: Identifying and rewarding the top 3-5 high-value customers with personalized gift cards in celebration of Thanksgiving

```
In [73]: mycursor.execute("SELECT C.Customer_ID,C.First_Name,C.Last_Name,SUM(P.Amount) AS Total_spend_$ FROM orders O

         # Fetch the results
         result1_5 = mycursor.fetchall()

         # Convert results to a DataFrame
         result1_5 = pd.DataFrame(result1_5, columns=["Customer_ID", "First_Name", "Last_Name", "Total_spend_$"])

         # Print the results
         print(result1_5)

         # Visualization: Bar Chart for Total Spend
         plt.figure(figsize=(10, 6))
         result1_5.plot(x="Customer_ID", y="Total_spend_$", kind="bar", color='green', rot=0)
         plt.title("Top 10 Customers by Total Spend")
         plt.xlabel("Customer ID")
         plt.ylabel("Total Spend ($)")
         plt.show()
```

	Customer_ID	First_Name	Last_Name	Total_spend_$
0	39	Bette	Kutch	2815.0
1	45	Mercedes	Monahan	2342.0
2	21	Lois	Kuvalis	2293.0
3	64	Lillie	Hilpert	2120.0
4	5	Maudie	Wyman	2048.0
5	13	Winfield	Konopelski	2002.0
6	25	Judson	Reynolds	1960.0
7	35	Deja	Morar	1773.0
8	9	Brady	Schiller	1696.0
9	6	Lilian	Durgan	1646.0

```
<Figure size 1000x600 with 0 Axes>
```

Visualizing the output using bar chart



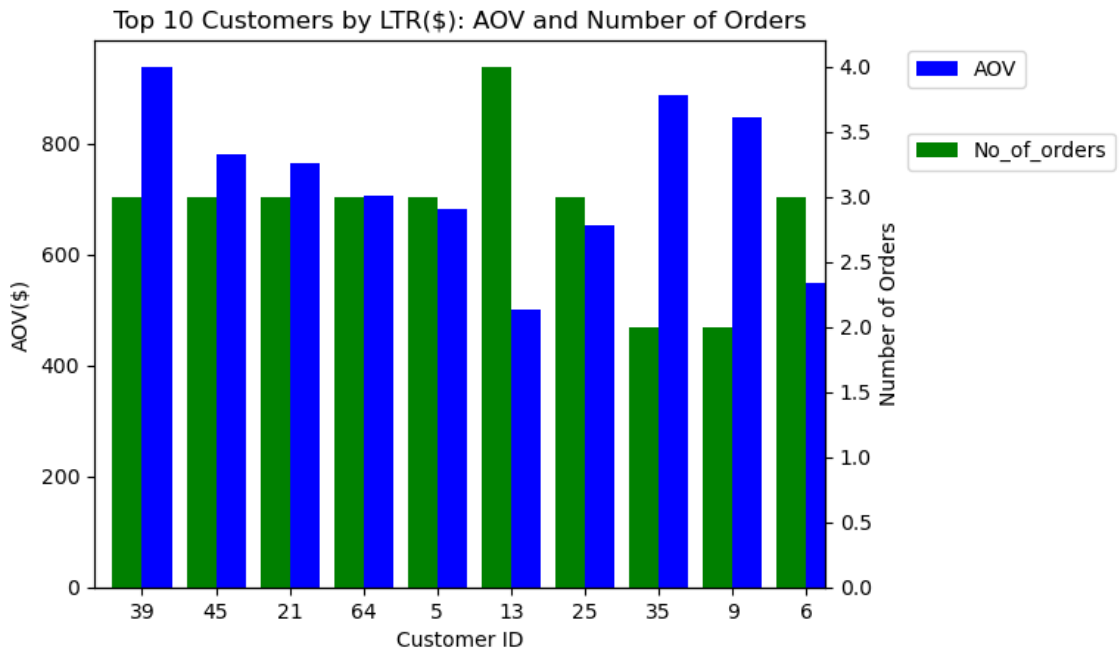
Find Top 10 High value customers and their average order value (AOV) & number of orders (OPC) using python

Analytical Purpose: Further analyzing the top-10 high LTR ($) customers to evaluate if the LTR is due to few high value orders or due to several low value orders or both

In [65]:
```
# Execute the query
mycursor.execute("SELECT Customer_ID, First_Name, Last_Name, Total_rev AS 'LTR($)', ROUND(AOV, 2) AS 'AOV($)', No_of_orders FROM

# Convert results to a DataFrame
result2 = pd.DataFrame(mycursor.fetchall(), columns=["Customer_ID", "First_Name", "Last_Name", "LTR($)", "AOV($)", "No_of_orders"

# Close cursor and connection
#mycursor.close()
#conn.close()

# Print the results
print(result2)

# Visualization: Bar Chart
plt.figure(figsize=(10, 6))
ax1 = result2.plot(x="Customer_ID", y="AOV($)", kind="bar", color='blue', rot=0, position=0, width=0.4)
ax2 = result2.plot(x="Customer_ID", y="No_of_orders", kind="bar", color='green', rot=0, position=1, width=0.4, secondary_y=True,

ax1.set_title("Top 10 Customers by LTR($): AOV and Number of Orders")
ax1.set_xlabel("Customer ID")
ax1.set_ylabel("AOV($)")
ax2.set_ylabel("Number of Orders")
# Place the legend outside the chart area
ax1.legend(["AOV"], loc="upper left", bbox_to_anchor=(1.1, 1))
ax2.legend(["No_of_orders"], loc="upper left", bbox_to_anchor=(1.1, 0.85))

plt.show()
```

```
   Customer_ID First_Name   Last_Name  LTR($)  AOV($)  No_of_orders
0           39      Bette       Kutch  2815.0  938.33             3
1           45   Mercedes     Monahan  2342.0  780.67             3
2           21       Lois     Kuvalis  2293.0  764.33             3
3           64     Lillie     Hilpert  2120.0  706.67             3
4            5     Maudie       Wyman  2048.0  682.67             3
5           13   Winfield  Konopelski  2002.0  500.50             4
6           25     Judson    Reynolds  1960.0  653.33             3
7           35       Deja       Morar  1773.0  886.50             2
8            9      Brady    Schiller  1696.0  848.00             2
9            6     Lilian      Durgan  1646.0  548.67             3
```

Visualizing the output using bar chart



Summary

In this project, the database designed on MySQL is an industry ready relational database which can be implemented in the retail market. This use case illustrated how to construct a database in MySQL and NoSQL, along with queries to analyze the data in the database. Python was used to create certain data visualizations for better insights and trends. In a real-world scenario, we can use our database to store data and analyze store performance. Additionally, we can employ customer analytics to understand customer behavior and adjust marketing strategies accordingly. We can also understand the sales trends across years and check for any seasonality based on which we can start any campaigns to boost sales in non-seasons.

This comprehensive approach ensures that the global superstore not only provides a seamless shopping experience for customers but also leverages data-driven insights for optimizing operations, enhancing customer satisfaction, and making informed business decisions.